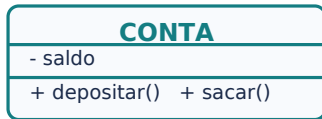


CLASSES, OBJETOS & ENCAPSULAMENTO



classe → objeto

Modelar entidades com estado e comportamento, protegendo detalhes internos da implementação.

- **Classe:** molde que define atributos e métodos
- **Objeto:** instância concreta criada a partir da classe
- **Atributo:** representa estado/dados do objeto
- **Método:** representa comportamento/operação
- **Construtor:** inicializa o estado do objeto
- **Encapsulamento:** restringe acesso ao estado interno
- **Visibilidade:** public, private, protected
- **Get/Set:** controlam leitura e alteração de atributos

Dica: Evite deixar atributos públicos sem necessidade. Encapsular permite validar regras e reduzir acoplamento.

HERANÇA, POLIMORFISMO & ABSTRAÇÃO



falar() → comportamentos diferentes

Reutilizar comportamento comum e trabalhar com diferentes implementações por uma mesma interface.

- **Herança:** classe filha reutiliza/estende a classe base
- **É-um:** relação típica para herança
- **Sobrescrita:** redefinir método herdado
- **Polimorfismo:** mesma chamada, comportamentos diferentes
- **Classe abstrata:** define estrutura comum e pode ter métodos abstratos
- **Interface:** contrato de operações esperadas
- **Un casting:** tratar objeto filho

Dica: Use herança quando houver relação “é-um” verdadeira. Para apenas reutilizar funcionalidade, composição costuma ser mais segura.

ASSOCIAÇÃO, COMPOSIÇÃO & UML



associação • composição

Representar como objetos colaboram e como as relações aparecem no modelo orientado a objetos.

- **Associação:** objetos conhecem/colaboram entre si
- **Agregação:** relação todo-parte fraca; partes podem existir sozinhas
- **Composição:** todo-parte forte; ciclo de vida das partes depende do todo
- **Dependência:** uso temporário de outra classe
- **Multiplicidade:** 1, 0..1, 1..*, 0..*
- **UML:** classe mostra nome, atributos e operações
- **Acoplamento:** dependência

Dica: Em UML, não desenhe relações só por aparência. Cada seta deve representar uma relação real no código ou no domínio.

EXCEÇÕES, GENÉRICOS & BOAS PRÁTICAS



robustez • reuso • manutenção

Construir código OO mais reutilizável, robusto e fácil de manter.

- **Exceção:** representa situação anormal durante a execução
- **try/catch/finally:** tratar erro e garantir limpeza de recursos
- **throw:** lançar explicitamente uma exceção
- **Genéricos:** reutilizam estruturas com segurança de tipos
- **SOLID:** princípios para reduzir acoplamento e facilitar evolução
- **SRP:** classe deve ter uma responsabilidade principal
- **OCP:** aberto para extensão, fechado para modificação
- **Boas práticas:** nomes claros,

Dica: OO não é só criar classes. O objetivo é distribuir responsabilidades de forma clara e tornar mudanças futuras menos custosas.